FPGA BASED IMAGE EDGE DETECTION FOR BRAIN TUMOUR ANALYSIS

PROJECT REPORT- PHASE 1

Submitted in partial fulfillment of the requirements for the award of the Degree of ***Bachelor of Technology*** *in* ***Electronics & Communication Engineering*** *of* ***APJ Abdul Kalam Technological University***

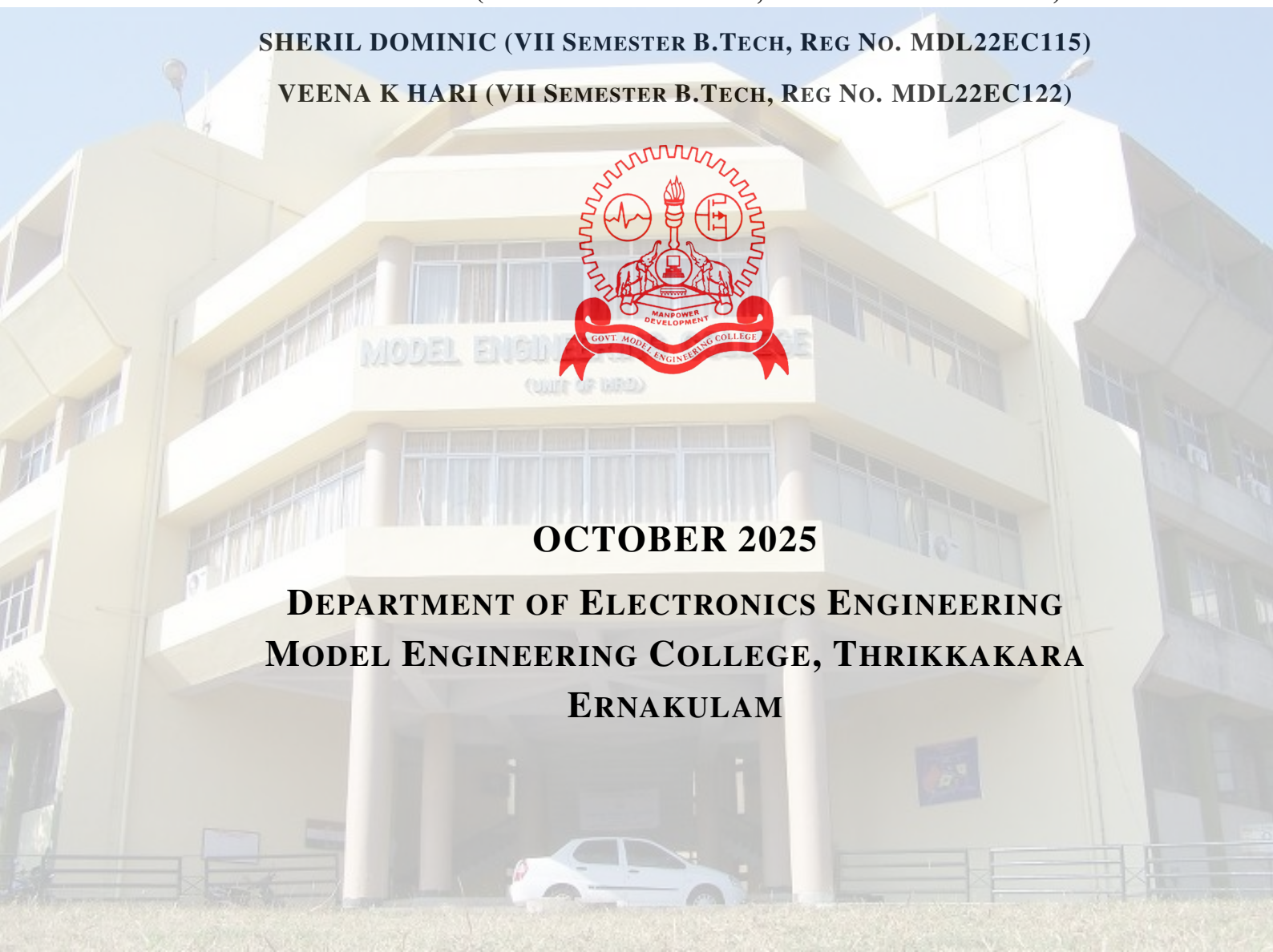
By

EMILTTA VARGHESE (VII SEMESTER B.TECH, REG NO. MDL22EC045)

HRIDHYA K P (VII SEMESTER B.TECH, REG NO. MDL22EC057)

SHERIL DOMINIC (VII SEMESTER B.TECH, REG NO. MDL22EC115)

VEENA K HARI (VII SEMESTER B.TECH, REG NO. MDL22EC122)



OCTOBER 2025

DEPARTMENT OF ELECTRONICS ENGINEERING

MODEL ENGINEERING COLLEGE, THRIKKAKARA

ERNAKULAM

MODEL ENGINEERING COLLEGE, THRIKKAKARA

DEPARTMENT OF ELECTRONICS ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"FPGA BASED IMAGE EDGE DETECTION FOR BRAIN TUMOUR ANALYSIS"** *is a bonafide record of the project work done by* **EMILTTA VARGHESE** *(VII Semester B.Tech, Reg No. MDL22EC045)* **HRIDHYA K P** *(VII Semester B.Tech, Reg No. MDL22EC057)* **SHERIL DOMINIC** *(VII Semester B.Tech, Reg No. MDL22EC115)* **VEENA K HARI** *(VI Semester B.Tech, Reg No. MDL22EC122) towards the partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering of APJ Abdul Kalam Technological University during the year 2025.*

PROJECT CO-ORDINATOR

MR. JAYADAS C K
ASSOCIATE PROFESSOR
DEPT. OF ELECTRONICS ENGG.

PROJECT CO-ORDINATOR

MRS. SHEEBA P S
ASSISTANT PROFESSOR
DEPT. OF ELECTRONICS ENGG.

PROJECT GUIDE

MRS. REMYA S
ASSISTANT PROFESSOR
DEPT. OF ELECTRONICS ENGG.

HEAD OF DEPARTMENT

DR. JOBYMOL JACOB
PROFESSOR
DEPT. OF ELECTRONICS ENGG.

ACKNOWLEDGEMENT

First of all, I would like to thank the ***Lord Almighty*** *who helped me to finish this project on time.*

I express my sincere thanks to ***Dr. Jayachandran E.S.****, The Principal, Model Engineering College, Thrikkakara, for providing opportunity and the environment to do the project in my college.*

I sincerely thank ***Dr. Jobymol Jacob****, Head of the Department, Dept. of Electronics, for his encouragement and constant support in making project successful.*

I would like to thank my class coordinator ***Mrs. Sheeba P.S.*** *, Asst. Professor, Dept. of Electronics, for giving me timely instruction,for the completion the work.*

I would like to thank my project coordinator ***Mr. Jayadas C.K.*** *, Associate Professor, Dept. of Electronics, for giving me technical advice, without which I could never been able to complete the work in time.*

I also wish to thank my project guide ***Mrs. Remya S****, Assistant Professor, Dept. of Electronics, for providing valuable guidance.*

An excellent group of teaching and non-teaching staff helped me for this project. I owe much the assistance they gave me while doing the project.

Last, but not least I would like to thank my parents and friends for all the moral support and that they have given me.

Emiltta Varghese (MDL22EC045)

Hridhya K P (MDL22EC057)

Sheril Dominic (MDL22EC115)

Veena K Hari (MDL22EC122)

ABSTRACT

Medical image processing has become an essential tool in modern healthcare, particularly for the early detection and diagnosis of brain tumours. Magnetic Resonance Imaging (MRI) is widely used because of its ability to provide high-resolution images of soft brain tissues. However, accurately identifying tumour boundaries from MRI scans is a challenging task due to the presence of noise, low contrast, and intensity inhomogeneities. Edge detection serves as a fundamental preprocessing step that helps in highlighting anatomical structures and abnormal regions by detecting intensity variations. Enhancing the accuracy and reliability of edge detection can significantly improve subsequent processes such as segmentation, analysis, and clinical interpretation.

The proposed system integrates contrast enhancement using CLAHE and noise reduction through bilateral filtering, followed by adaptive selection among Sobel, Prewitt, and Canny edge detection algorithms. Unlike traditional fixed-operator approaches, the system dynamically chooses the most suitable edge detector based on image characteristics, improving accuracy under varying imaging conditions. The design is targeted for FPGA implementation to achieve parallel processing, reduced latency, and real-time performance. This Phase-1 work focuses on system design, algorithm development, and architectural planning, forming a foundation for future hardware deployment and advanced medical image analysis.

Contents

List of Figures vi

List of Abbreviations vi

1 INTRODUCTION **1**

1.1 Background of the Project 1

1.2 Motivation 2

1.3 Importance of the problem 3

1.4 Objective and Scope 3

2 LITERATURE REVIEW **5**

3 PROBLEM STATEMENT AND PROPOSED SOLUTION **7**

3.1 Problem Statement 7

3.2 Proposed Solution 8

4 BLOCK DIAGRAM AND EXPLANATION **9**

4.1 Block Diagram 9

4.2 Block Diagram Explanation 9

5 FLOWCHART AND EXPLANTION **12**

5.1 Flowchart 12

5.2 Explanation 13

6 COMPONENTS USED **15**

6.1 Hardware Components 15

6.2 Software Components 17

7 IMPLEMENTATION AND DESIGN **18**

7.1 Development Environment Setup 18

7.2 FPGA-Based Edge Detection Architecture 18

7.2.1 Line Buffer Design 19

7.2.2 Sobel Edge Detection Module 19

7.2.3 Prewitt Edge Detection Module 20

7.2.4 Thresholding Module 20

7.2.5 Adaptive Selection Logic 20

7.3 Top-Level Adaptive Edge Detector Integration 21

7.4 Simulation and Verification 21

7.5 Final Implementation Outcome 22

8 EXPLANATION OF CODE **23**

8.1 Adaptive Edge Detection 23

9 TESTING AND RESULTS **25**

9.1 Testing Procedure 25

9.1.1 Python Testing 25

9.2 Observations and Output 26

9.2.1 Python Output Observations 26

9.3 Performance Analysis 30

10 APPLICATIONS,LIMITATIONS AND FUTURE SCOPE **32**

10.1 Applications 32

10.2 Limitations 32
10.3 Future Scope 32
10.4 Advantages 33

11 CONCLUSION **34**

Bibliography **35**

Appendices **36**

A Coding **37**

B Work Division **43**

List of Figures

Fig. 4.1 Block Diagram 9

Fig. 5.1 Flow Chart 12

Fig. 6.1 PYNQ Z2 FPGA Developement Board 15

Fig. 6.2 Enter Caption 16

Fig. 6.3 USB/JTAG interface for FPGA programming 16

Fig. 9.1 Original 27

Fig. 9.2 Bilateral Filtering 28

Fig. 9.3 Low Pass Filter 28

Fig. 9.4 High Pass Filter 29

Fig. 9.5 Edge Detected Output 30

List of Abbreviations

CPU	Central Processing Unit
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
HPF	High Pass Filter
IDE	Integrated Development Environment
LPF	Low Pass Filter
RAM	Random Access Memory
RGB	Red Green Blue
ROI	Region of Interest

Chapter 1

INTRODUCTION

Medical image processing is widely used to assist doctors in diagnosing serious diseases like brain tumours. MRI provides detailed images of brain tissues, but tumour boundaries are often unclear due to noise and low contrast. This project focuses on improving tumour boundary detection using an adaptive FPGA-based edge detection system.

1.1 Background of the Project

Brain tumours are abnormal and uncontrolled growths of cells within the brain that can interfere with normal brain functions such as movement, speech, memory, and sensory processing. If not detected and treated at an early stage, they may lead to severe neurological complications and can become life-threatening. Magnetic Resonance Imaging (MRI) is one of the most widely used diagnostic tools for brain tumour detection because it provides high-resolution images of soft tissues without exposing patients to harmful radiation. MRI scans allow doctors to observe internal brain structures in detail; however, the images often suffer from issues such as noise, low contrast, intensity inhomogeneity, and blurred boundaries between healthy and abnormal tissues, making precise tumour boundary identification a complex task.

Edge detection is a key image processing technique used to identify object boundaries by detecting sharp changes in pixel intensity values. In the context of medical imaging, accurate edge detection plays an important role in highlighting tumour margins, anatomical structures, and abnormal regions that assist in further processes such as segmentation and analysis. Conventional edge detection methods implemented in software can be computationally expensive and may not deliver real-time performance, especially when processing high-resolution MRI images. Additionally, the effectiveness of a single edge detection operator may vary depending

on image quality and noise levels. These challenges emphasize the need for an adaptive and hardware-accelerated solution capable of improving edge detection accuracy while maintaining high processing speed, which forms the foundation of this project.

1.2 Motivation

Accurate identification of brain tumour boundaries from MRI scans is essential for effective diagnosis, treatment planning, and monitoring of disease progression. However, MRI images often suffer from noise, low contrast, and intensity inhomogeneity, which make tumour regions difficult to distinguish clearly. As a result, radiologists must spend significant time carefully examining scans, and subtle tumour margins may still be missed.

With the advancement of medical imaging technologies, there is an increasing need for intelligent image processing systems that enhance critical image features while maintaining fast processing speed. An adaptive and hardware-accelerated approach can improve the reliability of edge detection across different image conditions and support real-time medical imaging applications, ultimately assisting clinicians in making more accurate decisions.

- MRI images often contain noise, low contrast, and intensity variations that obscure tumour boundaries, making accurate visual interpretation difficult and increasing the need for enhanced edge detection techniques.
- Traditional edge detection methods rely on fixed operators that may work well for certain images but fail under different imaging conditions, highlighting the importance of an adaptive multi-operator approach.
- Software-based image processing systems struggle to deliver real-time performance for high-resolution medical images, whereas FPGA-based hardware acceleration enables faster computation through parallel processing.
- Modern healthcare demands intelligent automated tools that improve diagnostic accuracy while reducing analysis time, and a real-time adaptive edge detection system can effectively support this need.

1.3 Importance of the problem

Accurate detection of brain tumour boundaries is crucial for effective diagnosis, treatment planning, and monitoring of disease progression. MRI images provide detailed views of brain tissues, but the presence of noise, low contrast, and intensity variations often makes tumour regions difficult to distinguish from surrounding healthy tissues. If tumour edges are not clearly identified, it may lead to incorrect estimation of tumour size, shape, and location, which can directly affect medical decisions. Therefore, improving edge detection in MRI images is an important step toward enhancing the reliability and efficiency of medical image analysis systems.

- Brain tumours can be life-threatening, and early as well as accurate detection plays a major role in improving patient survival rates and treatment outcomes.
- Clear identification of tumour boundaries is essential for planning surgeries, radiation therapy, and other medical treatments with higher precision and reduced risk.
- Noise and poor contrast in MRI images reduce the effectiveness of conventional edge detection methods, increasing the need for improved and adaptive techniques.
- Real-time image processing is increasingly important in modern healthcare systems, where faster analysis can support timely clinical decision-making.
- Hardware-accelerated systems such as FPGA-based solutions can provide both speed and accuracy, making them suitable for advanced medical imaging applications.

1.4 Objective and Scope

The main aim of this project is to design and develop an adaptive edge detection system that enhances brain tumour boundaries in MRI images using FPGA-based hardware acceleration. The system is intended to improve edge detection accuracy under different image conditions while achieving high processing speed suitable for real-time medical applications. This

phase focuses on algorithm development, system design, and planning for hardware implementation.

- To implement multiple edge detection algorithms such as Sobel, Prewitt, and Canny to effectively detect tumour boundaries in brain MRI images.
- To improve image quality using preprocessing techniques like Contrast Limited Adaptive Histogram Equalization (CLAHE) and bilateral filtering for better edge visibility.
- To design an adaptive selection mechanism that chooses the most suitable edge detection operator based on local image characteristics.
- To develop a system architecture targeted for FPGA implementation, enabling parallel processing and real-time performance.
- To validate the performance of the proposed approach using MRI images while limiting Phase 1 work to algorithm design, simulation, and architectural planning.
- To keep the system scalable so that future phases can include tumour segmentation, classification, and integration with AI-based medical diagnostic systems.

Chapter 2

LITERATURE REVIEW

Medical image processing has become an essential tool in the diagnosis and analysis of neurological disorders, particularly brain tumours. Magnetic Resonance Imaging (MRI) is widely used due to its high spatial resolution and ability to distinguish soft tissues. However, accurate interpretation of MRI images is often affected by noise, low contrast, and intensity variations. Edge detection plays a crucial role in highlighting structural boundaries and tumour regions, serving as a fundamental preprocessing step for segmentation and analysis.

Several classical edge detection algorithms such as Sobel, Prewitt, Roberts, Laplacian, and Canny have been extensively studied in image processing literature. Sobel and Prewitt operators are gradient-based methods that are computationally simple and suitable for real-time applications, but they are sensitive to noise and may produce thick edges. The Canny edge detector, although computationally more complex, provides thin and continuous edges with better noise suppression, making it more suitable for medical imaging applications. However, its complexity poses challenges for real-time hardware implementation.

Anis BenHajyoussef and Asma Saidani (2024) presented a comprehensive review of recent advances in image edge detection, covering classical, hybrid, and deep learning-based approaches. Their study highlighted the effectiveness of traditional edge detectors for basic boundary extraction while identifying gaps in real-time implementation and hardware optimization. Although the review provided valuable insights, it lacked experimental validation and hardware-based performance analysis.

Yueer Meng (2022) investigated FPGA-based implementations of multiple edge detection algorithms including Roberts, Laplacian, Prewitt, Sobel, and Canny. The study demonstrated that FPGA parallelism significantly improves processing speed, with Sobel providing better edge continuity and detail under hardware constraints. However, the work did not explore adaptive operator selection or advanced preprocessing techniques to handle varying noise levels

in medical images.

S. Taslimi et al. (2020) proposed an adaptive edge detection technique implemented on FPGA, where image histograms were analyzed to select appropriate mask sizes based on contrast variations. The results showed improved real-time edge detection compared to fixed-mask approaches. Despite its advantages, the design required higher FPGA resources, limiting scalability for complex medical images.

W. He and K. Yuan (2008) focused on improving the Canny edge detector and its FPGA realization using parallel processing techniques. Their implementation achieved reduced delay and real-time performance, demonstrating the feasibility of complex edge detection algorithms on hardware. However, the design complexity and high resource usage remained major challenges.

In the context of medical imaging, V. Sivakumar et al. (2020) proposed a modified segmentation method for brain tumour detection using enhanced edge detection techniques. Their approach improved tumour boundary identification in MRI images, emphasizing the importance of accurate edge detection in medical diagnosis. However, the method involved high computational complexity and increased resource utilization for large datasets.

M. Sonka et al. (2014) provided a comprehensive reference on image processing and machine vision, detailing mathematical models and practical techniques for edge detection, segmentation, and analysis. While the book offers strong theoretical foundations, it places limited emphasis on hardware-based or real-time FPGA implementations.

From the reviewed literature, it is evident that while numerous edge detection techniques exist, no single algorithm performs optimally under all imaging conditions. Software-based solutions often fail to meet real-time requirements, and fixed-operator hardware implementations lack adaptability to varying noise and contrast levels in MRI images. These limitations highlight the need for an adaptive multi-operator FPGA-based edge detection system with dynamic preprocessing to improve accuracy, efficiency, and real-time performance in brain tumour analysis.

Chapter 3

PROBLEM STATEMENT AND PROPOSED SOLUTION

Medical image analysis is a crucial step in diagnosing brain tumours using MRI scans. Although MRI provides detailed images of brain structures, accurately identifying tumour boundaries remains difficult due to variations in image quality. This chapter describes the challenges in existing approaches and presents the proposed adaptive FPGA-based edge detection solution.

3.1 Problem Statement

Brain MRI images often contain noise, low contrast, and intensity inhomogeneity, which make tumour boundaries difficult to distinguish from surrounding healthy tissues. Traditional edge detection techniques are mostly software-based and rely on fixed operators, which do not perform consistently across different image conditions. These limitations affect the accuracy and speed of tumour boundary detection, especially in real-time medical applications.

- MRI images frequently suffer from noise and poor contrast, which obscure tumour edges and reduce the effectiveness of conventional edge detection methods.
- Fixed edge detection operators may work well for certain images but fail under varying imaging conditions, leading to inconsistent boundary detection.
- Advanced algorithms like Canny provide better edge quality but involve high computational complexity that increases processing time.
- Software-based systems struggle to meet real-time requirements when processing high-resolution medical images.
- Lack of hardware acceleration limits the speed and efficiency of current medical image processing systems.

3.2 Proposed Solution

To overcome the limitations of existing methods, this project proposes an adaptive multi-operator edge detection system designed for FPGA-based implementation. The system enhances MRI image quality through effective preprocessing and dynamically selects the most suitable edge detection operator based on image characteristics. By combining adaptive algorithm selection with hardware acceleration, the proposed approach aims to deliver accurate tumour boundary detection with real-time performance and efficient resource utilization.

- Preprocessing of MRI images using CLAHE to enhance contrast and bilateral filtering to reduce noise while preserving important edge details.
- Implementation of multiple edge detection algorithms, including Sobel, Prewitt, and Canny, to handle different image conditions effectively.
- Adaptive selection mechanism that analyzes local image characteristics to choose the most suitable edge detection operator.
- FPGA-based hardware architecture designed with parallel processing and pipelining to achieve high-speed, real-time performance.

  system design that can be extended in future phases for tumour segmentation, classification, and integration with intelligent medical imaging systems.

Chapter 4

BLOCK DIAGRAM AND EXPLANATION

4.1 Block Diagram

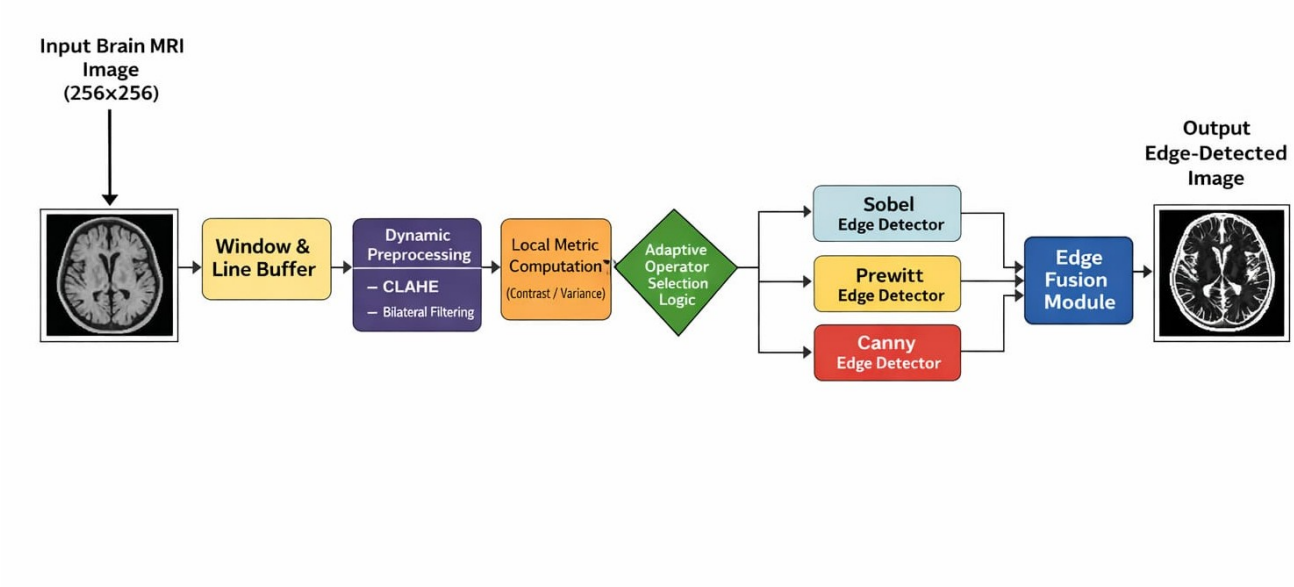


Figure 4.1: Block Diagram

4.2 Block Diagram Explanation

The block diagram of the proposed FPGA-based adaptive edge detection system for brain tumour analysis is shown in Figure 4.1. The system processes grayscale brain MRI images and generates an edge-detected output image by dynamically selecting the most suitable edge detection operator for each region of the image.

1. **Input Brain MRI Image**

The input to the system is a grayscale brain MRI image of size 256×256 pixels. These

images are obtained from a publicly available dataset (Kaggle) and resized to a fixed resolution to simplify hardware implementation and ensure uniform processing on the FPGA.

2. **Window and Line Buffer Module**

   The window and line buffer module stores incoming pixel streams and generates local neighborhood windows (typically 3×3) required for convolution-based image processing operations. This module enables real-time streaming of image data without requiring external memory access and efficiently supports edge detection kernels.

3. **Dynamic Preprocessing Module**

   The dynamic preprocessing module enhances the quality of the input image before edge detection. It consists of two sub-blocks:

   - **CLAHE (Contrast Limited Adaptive Histogram Equalization):** Improves local contrast in low-contrast regions of the MRI image, making tumour boundaries more distinguishable.
   - **Bilateral Filtering:** Reduces noise while preserving important edges, unlike conventional smoothing filters that blur edges.

   This adaptive preprocessing improves the robustness and accuracy of subsequent edge detection.

4. **Local Metric Computation Module**

   This module computes local image statistics such as contrast and variance for each image region or tile. These metrics provide information about noise levels and texture characteristics of the image and are used to guide adaptive operator selection.

5. **Adaptive Operator Selection Logic**

   Based on the computed local metrics, the adaptive operator selection logic determines the most suitable edge detection algorithm for each region of the image:

- Low noise and high contrast regions are processed using the Canny edge detector.
- Medium noise regions are processed using the Sobel edge detector.
- High texture or low contrast regions are processed using the Prewitt edge detector.

This adaptive selection improves edge detection performance across varying image characteristics while maintaining resource efficiency on the FPGA.

6. **Parallel Edge Detection Modules**

   The system includes three parallel edge detection modules implemented using Verilog HDL:

   - **Sobel Edge Detector:** Detects horizontal and vertical edges with moderate computational complexity.
   - **Prewitt Edge Detector:** A simpler operator suitable for noisy or textured regions.
   - **Canny Edge Detector:** Produces thin and continuous edges with high accuracy.

   Only the selected operator is activated for a given image region, reducing FPGA resource utilization.

7. **Edge Fusion Module**

   The edge fusion module combines the outputs from different edge detectors and adjacent image tiles. Weighted fusion based on local variance ensures smooth and continuous edges while avoiding artifacts at tile boundaries.

8. **Output Edge-Detected Image**

   The final output is an 8-bit grayscale edge-detected image of the same size as the input (256×256 pixels). The output highlights anatomical boundaries and tumour regions clearly, aiding further analysis and segmentation tasks.

Chapter 5

FLOWCHART AND EXPLANTION

5.1 Flowchart

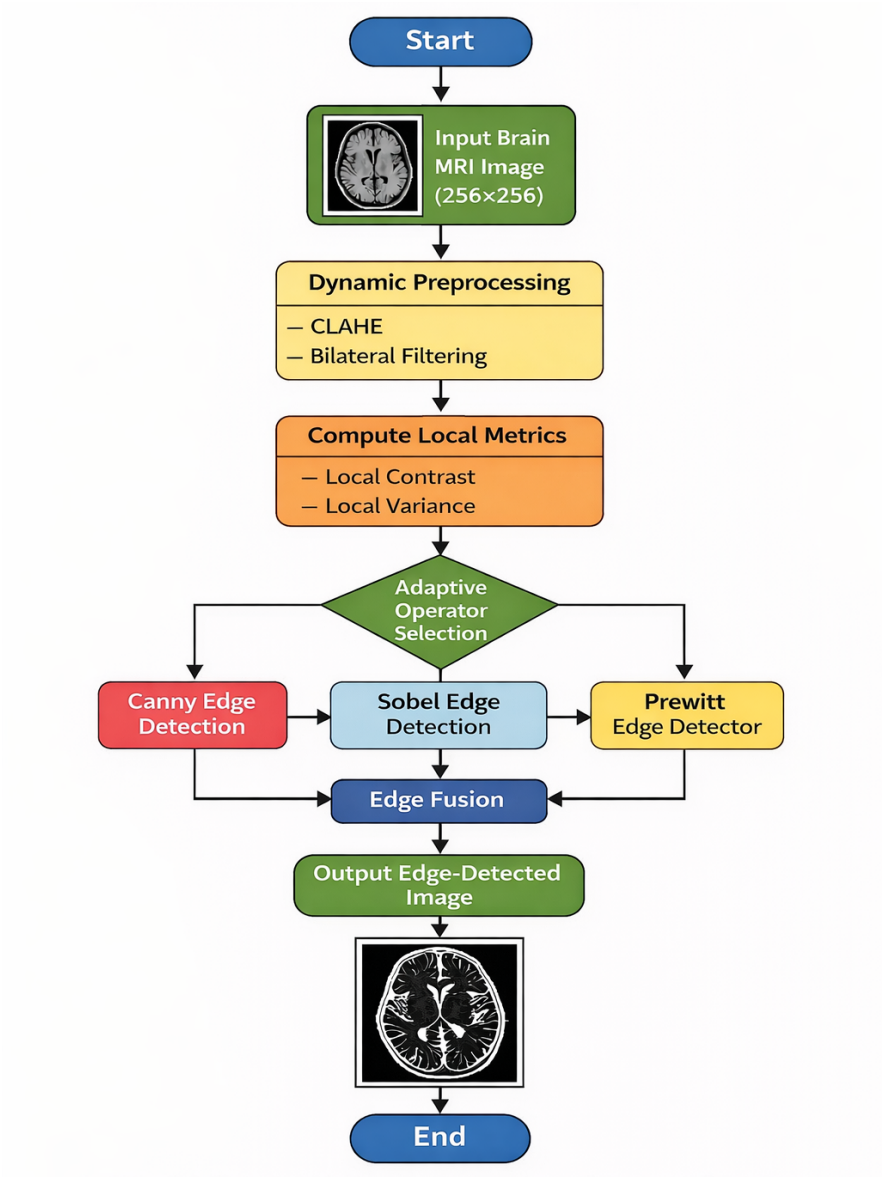


Figure 5.1: Flow Chart

5.2 Explanation

1. **Start**

   This block indicates the beginning of the system operation. All system resources and parameters required for image processing are initialized before execution.

2. **Input Brain MRI Image**

   In this stage, the brain MRI image is given as input to the system. The input image contains structural and tissue information of the brain, which is necessary for detecting tumour boundaries.

3. **Dynamic Preprocessing**

   Preprocessing enhances the quality of the input image and prepares it for accurate edge detection. It consists of:

   - CLAHE (Contrast Limited Adaptive Histogram Equalization): Improves local contrast and highlights fine details.
   - Bilateral Filtering: Reduces noise while preserving important edge information.

4. **Compute Local Metrics**

   This block analyzes the local characteristics of the image to support adaptive processing.

   - Local Contrast: Measures intensity variations in local regions.
   - Local Variance: Indicates texture complexity and edge strength.

   These metrics help determine the most suitable edge detection operator.

5. **Adaptive Operator Selection**

   Based on the computed local metrics, the system adaptively selects the appropriate edge detection method. This avoids using a single fixed operator for all image regions and improves detection accuracy.

- Canny Edge Detection

  Canny edge detection is applied to regions that require strong noise suppression and precise edge localization. It produces thin and continuous edges.

- Sobel Edge Detection

  The Sobel operator detects horizontal and vertical edges using gradient-based masks. It is computationally efficient and suitable for moderate edge variations.

- Prewitt Edge Detector

  The Prewitt operator is used for simple gradient-based edge detection. It detects basic edge structures with low computational complexity.

6. **Edge Fusion**

   In this stage, the edge maps obtained from different operators are combined. Edge fusion enhances edge continuity and provides a more robust and accurate edge representation of tumour boundaries.

7. **Output Edge-Detected Image**

   The final fused edge-detected image is produced as output. This image clearly highlights the edges and boundaries, which can assist in further medical analysis and diagnosis.

8. **End**

   This block indicates the completion of the processing. The system terminates after generating the final output image

Chapter 6

COMPONENTS USED

6.1 Hardware Components

PYNQ FPGA Development Board

The PYNQ Z2 FPGA development board, based on the Zynq-7000 System-on-Chip (SoC), serves as the core hardware platform for the proposed system. It integrates programmable logic with an embedded processing system, enabling parallel processing and pipelined execution of image preprocessing and edge detection algorithms. The board is used to implement CLAHE, bilateral filtering, adaptive operator selection, and Sobel, Prewitt, and Canny edge detection modules using Verilog HDL.

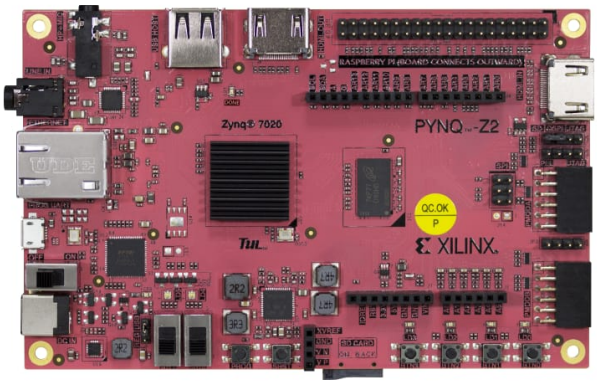


Figure 6.1: PYNQ Z2 FPGA Developement Board

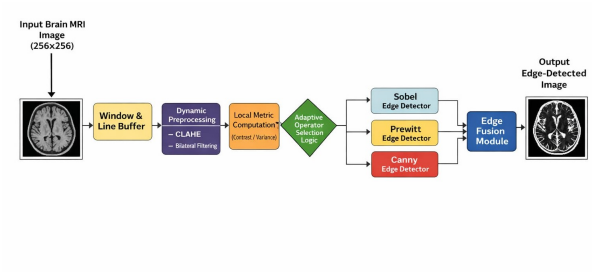


Figure 6.2: Enter Caption

Personal Computer (PC)

A personal computer is used for algorithm development, simulation, synthesis, and programming of the FPGA board. It hosts the required software tools such as Python and Xilinx Vivado and acts as the primary interface between the user and the hardware platform.

USB / JTAG Programming Cable

A USB/JTAG programming cable is used to establish communication between the personal computer and the PYNQ FPGA board. It enables bitstream download, configuration of the FPGA, and debugging during hardware testing.

Figure 6.3: USB/JTAG interface for FPGA programming

Power Supply Unit

A regulated power supply unit provides the necessary operating voltage to the PYNQ FPGA development board, ensuring stable and reliable operation during implementation and experimentation.

6.2 Software Components

Python

Python is used for algorithm development and software-level validation of image preprocessing and edge detection techniques. CLAHE, bilateral filtering, and Sobel, Prewitt, and Canny edge detection algorithms are first implemented and verified in Python to ensure correctness before hardware implementation.

Verilog HDL

Verilog Hardware Description Language (HDL) is used to design and implement the complete FPGA architecture. Individual modules such as line buffers, preprocessing blocks, local metric computation, adaptive operator selection logic, edge detection units, and edge fusion modules are implemented in Verilog to exploit FPGA parallelism.

Xilinx Vivado Design Suite

Xilinx Vivado Design Suite is used for synthesis, implementation, timing analysis, and bitstream generation. It also provides tools for functional simulation and verification of the designed hardware modules.

Vivado Simulator

Vivado Simulator is used to perform functional simulation and waveform analysis of individual Verilog modules and the integrated system prior to hardware deployment.

Chapter 7

IMPLEMENTATION AND DESIGN

7.1 Development Environment Setup

The implementation of the proposed adaptive edge detection system was carried out using FPGA design tools and simulation environments. Xilinx Vivado Design Suite was used for Verilog coding, simulation, synthesis, and verification of the proposed architecture.

- Vivado wasused to create and manage the FPGA project, design sources, and simulation sources.
- Behavioral simulation was performed to validate functional correctness before hardware deployment.
- Python was used as a reference platform to verify the correctness of edge detection outputs.

7.2 FPGA-Based Edge Detection Architecture

The adaptive edge detection system was implemented as a modular pipeline consisting of preprocessing, edge detection, thresholding, and adaptive selection logic. The design follows a streaming architecture suitable for real-time image processing applications. The major modules implemented are:

- Line Buffer (3×3 window generation)
- Sobel Edge Detection Module
- Prewitt Edge Detection Module

- Thresholding Module
- Adaptive Selector Module
- Top-Level Adaptive Edge Detector

7.2.1 Line Buffer Design

A 3×3 line buffer was implemented to store incoming pixel streams and generate neighborhood windows required for convolution-based edge detection algorithms.

- Two internal row buffers were used to store previous pixel rows.
- A column counter controlled pixel indexing and window formation.
- Proper reset initialization and boundary handling were implemented to avoid undefined values during simulation.
- The line buffer introduces pipeline latency, which is expected in streaming image architectures.

This module ensures continuous and synchronized data flow to the edge detection operators.

7.2.2 Sobel Edge Detection Module

The Sobel operator was implemented using horizontal and vertical convolution masks to detect edges with higher sensitivity to noise.

- Horizontal (Gx) and vertical (Gy) gradients were computed using fixed-point arithmetic.
- A hardware-efficient gradient magnitude approximation ($|Gx|$ + $|Gy|$) was used instead of square root operations.
- The module operates fully in combinational logic, making it suitable for high-speed FPGA implementation.
- Simulation results were compared with Python-based Sobel outputs for validation.

7.2.3 Prewitt Edge Detection Module

The Prewitt edge detection module was implemented as a low-complexity alternative to the Sobel operator.

- Horizontal and vertical gradients were computed using simpler convolution kernels.
- Fixed-point arithmetic was used to minimize hardware resource usage.
- The output gradient magnitude was generated using absolute sum approximation.
- Functional correctness was verified through simulation and comparison with Python reference results.

7.2.4 Thresholding Module

The thresholding module converts gradient magnitude values into a binary edge map.

- A programmable threshold value was provided as input.
- Gradient values greater than the threshold were classified as edges.
- The output was generated as an 8-bit binary signal (0 or 255) for easy visualization and further processing.
- This module ensures noise suppression and clear edge representation.

7.2.5 Adaptive Selection Logic

An adaptive selector module was designed to dynamically choose the appropriate edge detection operator based on image contrast.

- Local contrast information was provided as an input parameter.
- Prewitt operator was selected for medium contrast regions.
- Sobel operator was selected for high contrast regions.

- Only one edge detection output is enabled at a time to optimize hardware resources.

This adaptive approach improves edge detection accuracy under varying image conditions.

7.3 Top-Level Adaptive Edge Detector Integration

All sub-modules were integrated into a top-level module named adaptive edge detector.

- The line buffer feeds pixel neighborhoods to both Sobel and Prewitt operators.
- Thresholding is applied independently to each operator output.
- The adaptive selector determines the final edge output.
- Proper synchronization was maintained across pipeline stages to ensure correct timing.

System-level simulation confirmed correct end-to-end data flow and stable output behavior.

7.4 Simulation and Verification

Behavioral simulation was performed using Vivado to verify the complete adaptive edge detection pipeline.

- Clock and reset signals were applied to initialize the system.
- Pixel streams were provided to simulate real-time image input.
- Initial pipeline latency was observed during line buffer filling.
- After warm-up, valid binary edge outputs were generated.
- undefined (X) values were observed after proper initialization.

The simulation results confirmed correct adaptive operation and functional accuracy of the design.

7.5 Final Implementation Outcome

The final implementation successfully demonstrates an FPGA-based adaptive edge detection system capable of dynamically switching between Sobel and Prewitt operators based on contrast information.

- Modular and scalable architecture
- Hardware-efficient fixed-point implementation
- Stable and verified simulation results
- Suitable for real-time image processing applications

Chapter 8

EXPLANATION OF CODE

8.1 Adaptive Edge Detection

This section explains the Python implementation of the adaptive edge detection algorithm used as a reference model for validating the FPGA-based system.

1. OpenCV (`cv2`), NumPy, and Matplotlib libraries are imported for image processing, numerical operations, and result visualization.
2. The input image is read in grayscale format using `cv2.imread()` to simplify edge detection processing.
3. The input image is resized to a fixed resolution of 256×256 pixels to ensure uniform processing and comparison with FPGA outputs.
4. Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied to enhance local contrast and improve visibility of edges in low-contrast regions.
5. Bilateral filtering is used to reduce noise while preserving important edge information.
6. Image contrast is computed using the standard deviation of pixel intensities to analyze overall image characteristics.
7. An appropriate edge detection operator is selected dynamically based on the computed contrast value:
    - Low contrast images use Canny edge detection.
    - Medium contrast images use Prewitt edge detection.
    - High contrast images use Sobel edge detection.

8. Sobel edge detection is implemented using horizontal and vertical convolution kernels to calculate gradient magnitude.
9. Prewitt edge detection is implemented using simpler convolution kernels for reduced computational complexity.
10. A simplified Canny edge detection algorithm is implemented, including smoothing, gradient calculation, non-maximum suppression, and double thresholding.
11. Manual two-dimensional convolution is performed to closely match FPGA-style convolution operations for validation purposes.
12. Gradient magnitude values are normalized to the range 0–255 to ensure consistent thresholding and visualization.
13. Thresholding is applied to suppress weak edges and retain only significant edges.
14. Sobel, Prewitt, and Canny edge outputs are combined using weighted fusion to improve edge continuity and robustness.
15. The preprocessed grayscale image and the final adaptive edge map are displayed using Matplotlib for visual analysis.
16. The Python implementation is used as a reference model to validate the correctness of the FPGA-based adaptive edge detection system.

Chapter 9

TESTING AND RESULTS

This chapter presents the testing procedure and results obtained from the proposed FPGA-based edge detection system. The system is verified in two stages: software-level verification using Python and hardware-level verification using Vivado simulation. The objective of this phase is to evaluate the effectiveness of preprocessing techniques and edge detection algorithms in a software environment.

9.1 Testing Procedure

This section outlines the procedure followed to test the proposed edge detection system using Python simulation. It includes preprocessing of the input image and application of Sobel, Prewitt, and Canny algorithms for performance evaluation.

9.1.1 Python Testing

The edge detection algorithms were implemented in Python to validate their theoretical and functional behavior before hardware realization.

1. Selection of Input Image: Standard test images containing objects with clear boundaries and varying intensity levels were selected.

2. Grayscale Conversion: Since edge detection relies on intensity variation, color images were converted into grayscale format to simplify processing and reduce computational complexity.

3. Application of Sobel and Prewitt Operators: Convolution masks were applied to compute horizontal and vertical gradients. These gradients were combined to generate the edge magnitude image.

4. Implementation of Canny Edge Detection: The Canny algorithm was executed in multiple stages:
   - Gaussian smoothing to reduce noise
   - Gradient magnitude and direction calculation
   - Non-maximum suppression to thin edges
   - Double thresholding to classify strong and weak edges
   - Hysteresis to track and preserve true edges
5. Result Storage and Analysis: The edge-detected outputs from each algorithm were stored and visually analyzed for clarity, thickness, and noise suppression.

   The Python outputs served as reference models for verifying the correctness of the Verilog implementation.

9.2 Observations and Output

This section presents the outputs obtained from testing the proposed edge detection system and the observations made from them. The performance of the system is evaluated based on preprocessing quality and edge detection accuracy. In Python testing, preprocessing outputs such as Bilateral filtering, Low Pass Filtering , and High Pass Filtering are analyzed before applying Sobel, Prewitt, and Canny operators. These results serve as reference outputs for validating the hardware implementation.

9.2.1 Python Output Observations

The Python implementation was used to analyze the behavior of the edge detection system in a software environment before hardware realization. This stage includes preprocessing operations such as Bilateral filtering, Low Pass Filtering (LPF), and High Pass Filtering (HPF) to enhance image quality and emphasize important features. The outputs from these preprocessing steps help in understanding how noise reduction and edge enhancement affect the final

results. After preprocessing, Sobel, Prewitt, and Canny algorithms were applied to detect object boundaries based on intensity variations. The following figures present the outputs obtained at each stage along with observations on their performance.

1. **Input Image**

   This figure shows the original grayscale input image used for testing the edge detection algorithms. The image contains objects with varying intensity levels, textures, and boundaries, making it suitable for evaluating edge detection performance. This image acts as the reference input for all preprocessing and edge detection operations. Clear visibility of object boundaries in the original image helps in comparing how effectively edges are extracted in later stages.

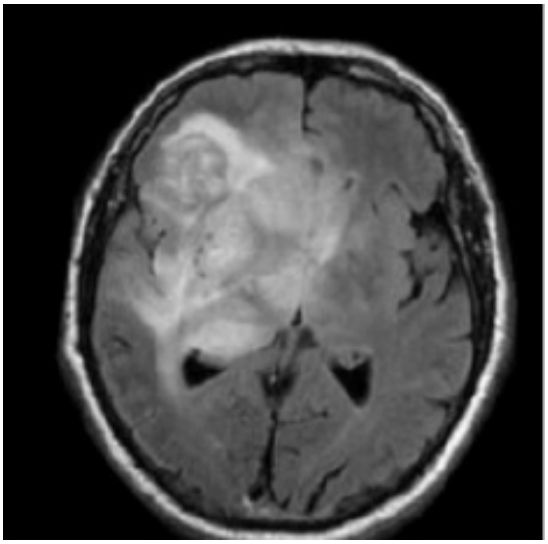


Figure 9.1: Original

2. **Bilateral Filtering** This figure presents the output of the bilateral filtering stage used for preprocessing. The bilateral filter smooths the image while preserving important edge information, unlike conventional linear filters that may blur edges. Noise and small intensity variations are reduced without affecting sharp boundaries. The output demonstrates improved visual smoothness while maintaining object contours, which helps in achieving more accurate edge detection in subsequent stages.

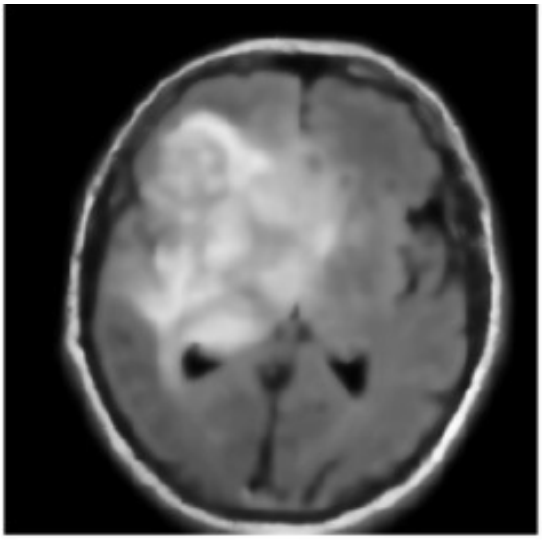


Figure 9.2: Bilateral Filtering

3. **Low Pass Filter (LPF) Output**

   This figure shows the result after applying a Low Pass Filter to the input image. The LPF reduces high-frequency noise components and smooths rapid intensity variations across the image. While this process improves noise immunity, some fine details and sharp edges become slightly blurred. The LPF output is useful for analyzing how smoothing affects edge sharpness before applying gradient-based edge detection algorithms.

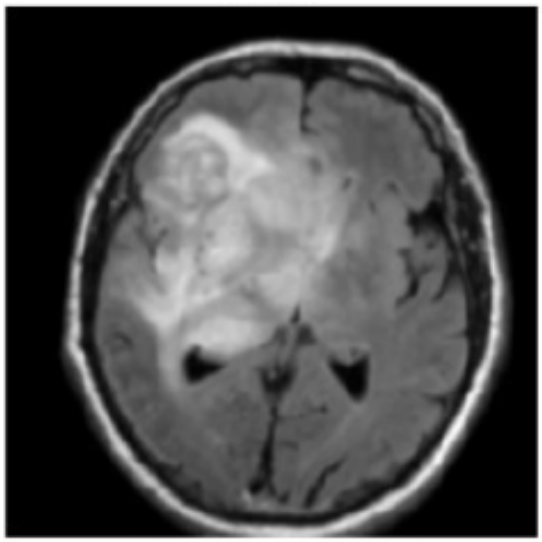


Figure 9.3: Low Pass Filter

4. **High Pass Filter (HPF) Output**:

   This figure illustrates the output of the High Pass Filter, which enhances high-frequency components such as edges and fine details. Unlike LPF, the HPF suppresses smooth regions and highlights sudden intensity changes. The edges of objects become more pronounced, making this stage helpful in emphasizing boundaries before applying edge detection operators. However, noise may also be amplified along with edges.

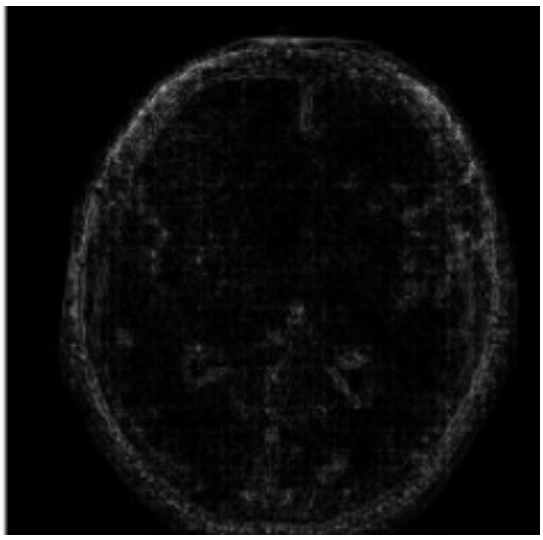


Figure 9.4: High Pass Filter

5. **Edge Detection Output (Sobel / Prewitt / Canny)**

   This figure shows the final edge detection result obtained after applying Sobel, Prewitt, or Canny algorithms to the preprocessed image. The detected edges represent boundaries of objects based on intensity gradients. Sobel and Prewitt produce gradient-based edges with moderate thickness, while Canny provides thinner and more continuous edges with better noise suppression. The output confirms that preprocessing improves the clarity and accuracy of detected edges.

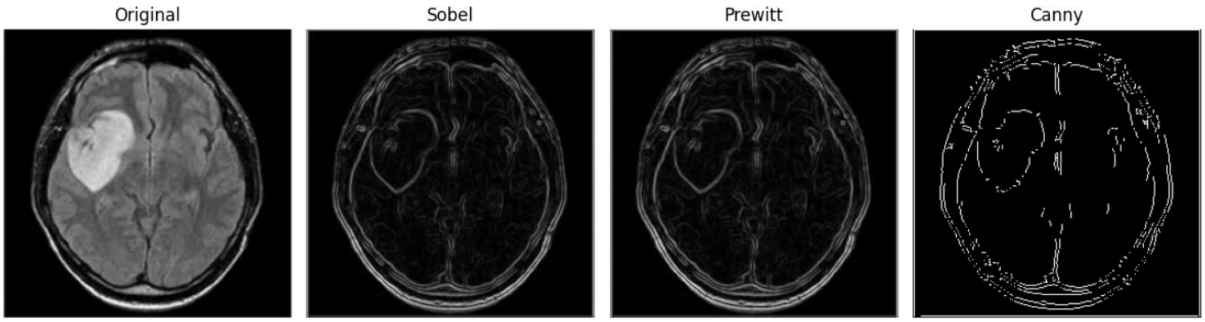


Figure 9.5: Edge Detected Output

9.3 Performance Analysis

The performance of the proposed edge detection system was evaluated based on both software and hardware simulations. This analysis includes the effect of preprocessing, quality of edge detection, and comparison between Python and Verilog results.

1. Edge Detection Quality

   - Sobel Operator: Sobel successfully detected strong horizontal and vertical edges in both Python and Verilog simulations. The edges are moderately thick and continuous, making it suitable for general edge detection tasks.
   - Prewitt Operator: Prewitt detected edges similarly to Sobel but with slightly lower gradient intensity. It provides simpler computation, which is advantageous for hardware implementation.
   - Canny Operator: Canny produced the thinnest and most precise edges with effective noise suppression. Its multi-stage process ensures accurate localization of object boundaries but involves more computation compared to Sobel and Prewitt.

2. Preprocessing Effect

   - Bilateral Filter: Reduces noise while preserving edges, resulting in cleaner edge detection for all algorithms.

- Low Pass Filter (LPF): Smooths the image and reduces high-frequency noise but slightly blurs fine edges.
- High Pass Filter (HPF): Enhances edges and fine details but may amplify noise alongside edges.

Chapter 10

APPLICATIONS,LIMITATIONS AND FUTURE SCOPE

10.1 Applications

1. Brain tumour detection and boundary identification in MRI images for medical diagnosis.
2. Preprocessing stage in medical image segmentation and classification systems.
3. Real-time medical imaging applications using FPGA-based hardware acceleration.
4. Academic and research projects related to medical image processing and FPGA design.
5. Integration into clinical decision-support systems for assisting radiologists.

10.2 Limitations

- Performance may degrade for highly noisy or low-contrast MRI images.
- Complex algorithms like Canny require higher FPGA resources compared to simpler operators.
- Limited to grayscale MRI images and does not include full tumour segmentation or classification.
- High-resolution image processing may increase processing time and hardware utilization.
- Requires careful hardware design and optimization for real-time performance.

10.3 Future Scope

1. Integration of machine learning or deep learning techniques for adaptive edge detection.

2. Extension of the system to include tumour segmentation and classification.
3. Support for real-time image acquisition using camera or medical imaging sensors.
4. Optimization for higher-resolution images using advanced FPGA architectures.
5. Integration with IoT-based medical monitoring and telemedicine systems.

10.4 Advantages

- Enables fast and real-time edge detection using FPGA parallel processing.
- Improves edge detection accuracy through adaptive multi-operator selection.
- Efficient utilization of hardware resources with pipelined architecture.
- Enhances visibility of tumour boundaries and brain structures.
- Suitable for real-time medical image processing applications.

Chapter 11

CONCLUSION

This project presents the design and implementation of an FPGA-based adaptive multi-operator edge detection system for brain tumour analysis using MRI images. The proposed system combines classical edge detection algorithms such as Sobel, Prewitt, and Canny with dynamic preprocessing techniques including CLAHE and bilateral filtering to enhance image quality and improve edge detection accuracy. By leveraging the parallel processing capability of FPGA hardware, the system achieves faster processing speeds suitable for real-time medical image analysis.

The successful implementation and verification of the edge detection algorithms demonstrate the effectiveness of adaptive operator selection in handling variations in noise and contrast present in MRI images. The use of FPGA-based architecture enables efficient resource utilization through pipelining and parallelism, addressing the limitations of software-based edge detection approaches in time-critical medical applications.

Although the current work focuses on edge detection as a preprocessing step, it establishes a strong foundation for advanced medical image analysis systems. With further integration of segmentation, classification, and machine learning techniques, the proposed system can be extended to support complete brain tumour diagnosis and decision-support applications. Overall, this project validates the feasibility of FPGA-based adaptive edge detection for medical imaging and contributes to the development of efficient, real-time, and reliable brain tumour analysis systems.

. . .

Bibliography

[1] A. BenHajyoussef and A. Saidani, "Recent advances on image edge detection," IntechOpen, 2024.

[2] Y. Meng, "Edge detection algorithm based on FPGA," Applied and Computational Engineering, 2022.

[3] S. Taslimi, R. Faraji, A. Aghasi, and H. R. Naji, "Adaptive edge detection technique implemented on FPGA," IEEE Transactions on Image Processing, 2020.

[4] W. He and K. Yuan, "An improved Canny edge detector and its realization on FPGA," in *Proceedings of the World Congress on Intelligent Control and Automation (WCICA)*, 2008.

[5] V. Sivakumar, R. Gopalakrishnan, and M. Rajesh, "A novel method for segmenting brain tumor using modified segmentation," *International Journal of Medical Imaging*, 2020.

[6] M. Sonka, V. Hlavac, and R. Boyle, *Image Processing, Analysis, and Machine Vision*, 4th ed. Boston, MA, USA: Cengage Learning, 2014.

[7] R. Gonzalez and R. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson Education, 2018.

[8] Xilinx Inc., *Vivado Design Suite User Guide*, San Jose, CA, USA, 2021.

Appendices

Appendix A

Coding

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# Helper Functions
# ----------------------------
def normalize(img):
    img = img - img.min()
    img = img / img.max() * 255
    return img.astype(np.uint8)

def convolve2d(img, kernel):
    kh, kw = kernel.shape
    h, w = img.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='edge')
    out = np.zeros_like(img, dtype=float)

    for i in range(h):
        for j in range(w):
            out[i, j] = np.sum(padded[i:i+kh, j:j+kw] * kernel)
    return out
```

```
# ----------------------------
# Edge Detection Functions
# ----------------------------
def sobel_edge(img, threshold=100):
    Kx = np.array([[1,0,-1],[2,0,-2],[1,0,-1]])
    Ky = np.array([[1,2,1],[0,0,0],[-1,-2,-1]])

    gx = convolve2d(img, Kx)
    gy = convolve2d(img, Ky)

    grad = np.hypot(gx, gy)
    grad = normalize(grad)
    grad[grad < threshold] = 0
    return grad

def prewitt_edge(img, threshold=100):
    Kx = np.array([[1,0,-1],[1,0,-1],[1,0,-1]])
    Ky = np.array([[1,1,1],[0,0,0],[-1,-1,-1]])

    gx = convolve2d(img, Kx)
    gy = convolve2d(img, Ky)

    grad = np.hypot(gx, gy)
    grad = normalize(grad)
    grad[grad < threshold] = 0
    return grad

def canny_edge(img, low=50, high=150):
```

```
kernel = np.array([[1,2,1],
                   [2,4,2],
                   [1,2,1]]) / 16
smoothed = convolve2d(img, kernel)

Kx = np.array([[1,0,-1],[2,0,-2],[1,0,-1]])
Ky = np.array([[1,2,1],[0,0,0],[-1,-2,-1]])

gx = convolve2d(smoothed, Kx)
gy = convolve2d(smoothed, Ky)

magnitude = np.hypot(gx, gy)
angle = np.rad2deg(np.arctan2(gy, gx)) % 180

Z = np.zeros_like(magnitude)
h, w = magnitude.shape

for i in range(1, h-1):
    for j in range(1, w-1):
        q = r = 0
        if (0 <= angle[i,j] < 22.5) or (157.5 <= angle[i,j] <= 180
            q, r = magnitude[i, j+1], magnitude[i, j-1]
        elif 22.5 <= angle[i,j] < 67.5:
            q, r = magnitude[i+1, j-1], magnitude[i-1, j+1]
        elif 67.5 <= angle[i,j] < 112.5:
            q, r = magnitude[i+1, j], magnitude[i-1, j]
        elif 112.5 <= angle[i,j] < 157.5:
            q, r = magnitude[i-1, j-1], magnitude[i+1, j+1]
```

```
            if magnitude[i,j] >= q and magnitude[i,j] >= r:
                Z[i,j] = magnitude[i,j]

    res = np.zeros_like(Z)
    strong, weak = 255, 75

    strong_i, strong_j = np.where(Z >= high)
    weak_i, weak_j = np.where((Z >= low) & (Z < high))

    res[strong_i, strong_j] = strong
    res[weak_i, weak_j] = weak

    for i in range(1, h-1):
        for j in range(1, w-1):
            if res[i,j] == weak:
                if 255 in res[i-1:i+2, j-1:j+2]:
                    res[i,j] = strong
                else:
                    res[i,j] = 0

    return res.astype(np.uint8)

# ----------------------------
# Preprocessing
# ----------------------------
def clahe_preprocessing(img):
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    return clahe.apply(img)
```

```
def bilateral_filter(img):
    return cv2.bilateralFilter(img, 9, 75, 75)

# ----------------------------
# Adaptive Operator Selection
# ----------------------------
def choose_edge_operator(img):
    contrast = img.std()
    if contrast < 50:
        return canny_edge
    elif contrast < 100:
        return prewitt_edge
    else:
        return sobel_edge

# ----------------------------
# MAIN (Single Image)
# ----------------------------
image_path = "Y7.jpg"   #     Your image file

img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Y7.jpg not found")

img = cv2.resize(img, (256,256))

# Preprocessing
img = clahe_preprocessing(img)
img = bilateral_filter(img)
```

```
# Adaptive selection
edge_func = choose_edge_operator(img)
edge_map = edge_func(img)

# Multi-operator fusion
edge_map = cv2.addWeighted(sobel_edge(img), 0.3,
                           prewitt_edge(img), 0.3, 0)
edge_map = cv2.addWeighted(edge_map, 1.0,
                           canny_edge(img), 0.4, 0)

# ----------------------------
# Display
# ----------------------------
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title("Preprocessed Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(edge_map, cmap='gray')
plt.title("Adaptive Edge Map")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Appendix B

Work Division

Member	Work Assigned (Phase 1)
Emiltta Varghese	**Preprocessing, Canny Edge Detection & Supporting Documentation:** Implemented preprocessing techniques including bilateral filtering, low pass filtering, and high pass filtering, developed the canny edge detection algorithm, analyzed multi stage behavior, and prepared clear documentation for the testing and results chapter.
Hridhya K P	**Python-Based Edge Detection Algorithms & Gradient Analysis:** Implemented sobel and prewitt edge detection algorithms in python, performed detailed gradient computation and edge magnitude analysis, using standard reference test datasets, and documented comparative observations for the testing and results chapter section.
Sheril Dominic	**Testing, Technical Analysis & Documentation Support:** Assisted in python based implementation and testing of preprocessing and edge detection algorithms, evaluated performance across multiple images for edge detection accuracy, supported validation, and contributed to refining the testing and results chapter.
Veena K Hari	**Python-Based Testing, Preprocessing Evaluation & Result Analysis:** Conducted python based testing of sobel prewitt and canny edge detection algorithms using selected images, evaluated preprocessing effects on edge clarity and noise suppression, and organized figures and outputs for the testing chapter.

Table B.1: Work Allocation Breakdown for Phase 1